Rapport de Projet – Développement Web ING1

Plateforme Maison Intelligente, performante et responsive

Groupe projet

Année universitaire 2025–2026

Membres du groupe

— Khayem Ben Ghorbel
— Rindra (nom complet à compléter)
— Ziyad (nom complet à compléter)
— Anastasia (nom complet à compléter)
— Valerie (nom complet à compléter)

Table des matières

1 Introduction et contexte **2**

2 Objectifs, enjeux et périmètre **2**
2.1 Objectifs fonctionnels . . . 2
2.2 Enjeux techniques . . . 2

3 Technologies et architecture **2**
3.1 Stack retenue . . . 2
3.2 Architecture logique . . . 2
3.3 Routes principales . . . 2

4 Méthodologie de gestion de projet **3**
4.1 Répartition des tâches . . . 3
4.2 Diagramme de Gantt (synthèse) . . . 3

5 Étapes réalisées (méthodologie de projet) **3**
5.1 Phase 1 – Planification . . . 3
5.2 Phase 2 – Conception . . . 4
5.3 Phase 3 – Développement itératif . . . 4
5.4 Phase 4 – Validation et stabilisation . . . 4

6 Couverture fonctionnelle par module **4**
6.1 Module Information . . . 4
6.2 Module Visualisation . . . 4
6.3 Module Gestion . . . 4
6.4 Module Administration . . . 4

7 UML amélioré **4**
7.1 Structure conceptuelle . . . 4
7.2 Cardinalités clés . . . 5

8 Schéma de base de données amélioré **5**
8.1 Choix de modélisation relationnelle 5
8.2 Tables principales 5
8.3 Améliorations proposées du schéma 5

9 Tests, performance, responsive, accessibilité **6**
9.1 Vérifications techniques 6
9.2 Responsive et UX 6
9.3 Accessibilité (base) 6

10 Déploiement et exploitation **6**
10.1 Render 6
10.2 Important pour la soutenance 6

11 Difficultés rencontrées et solutions **6**

12 Conclusion et perspectives **7**

1 Introduction et contexte

Le projet s'inscrit dans le cadre du module ING1 *Projet Développement Web 2025–2026*. Le sujet consiste à réaliser une plateforme intelligente autour de l'Internet des Objets, avec un thème libre et des modules fonctionnels imposés.

Nous avons choisi le thème **Maison Intelligente**. L'objectif est de centraliser :

— la consultation d'informations (objets, pièces, services),
— la visualisation pour les membres,
— la gestion opérationnelle avancée,
— l'administration de la plateforme.

Le projet répond aux contraintes principales du cahier des charges :

— utilisation de frameworks,
— base de données réelle,
— moteur de recherche avec au moins deux filtres,
— dépôt Git avec commits réguliers,
— design responsive, accessibilité, tests et optimisation.

2 Objectifs, enjeux et périmètre

2.1 Objectifs fonctionnels

1. **Module Information** (visiteur) : free tour et recherche d'informations.
2. **Module Visualisation** (utilisateur simple) : inscription, connexion, profil, consultation.
3. **Module Gestion** (utilisateur complexe) : pilotage des objets, statistiques, maintenance.
4. **Module Administration** (administrateur) : gouvernance utilisateurs et supervision globale.

2.2 Enjeux techniques

— Modéliser un domaine riche avec de nombreux types d'objets connectés.
— Assurer une cohérence front/back sur des comportements spécifiques (ex. machine à café, aspirateur, alarmes).
— Garantir une démonstration stable en soutenance (données seed réalistes et reproductibles).

3 Technologies et architecture

3.1 Stack retenue

— **Backend** : Java 21, Spring Boot 3.3, Spring Data JPA, Bean Validation.
— **Frontend** : React 19, Vite.
— **BDD** : H2 (mode fichier en développement), préparation MySQL.
— **Déploiement** : Render (API Docker + Frontend statique).

3.2 Architecture logique

— Couche **modèle** : entités métier (Maison, Pièce, Utilisateur, ObjetConnecte, etc.).
— Couche **repository** : accès aux données via Spring Data.
— Couche **service** : logique métier (points, sessions, scénarios).
— Couche **controller** : API REST par module.
— Couche **frontend** : composants UI modulaires, états locaux, appels API.

3.3 Routes principales

— `/api/info/*`

— `/api/auth/*`
— `/api/visualisation/*`
— `/api/gestion/*`
— `/api/admin/*`

4 Méthodologie de gestion de projet

Nous avons appliqué une démarche itérative inspirée de **SCRUM** :

— backlog priorisé (fichiers `PLAN.md` et `NEXT_TASKS.md`),
— livraisons incrémentales,
— vérifications régulières (builds, tests de parcours, smoke test).

4.1 Répartition des tâches

Membre	**Responsabilité principale**	**Contributions réalisées**
Khayem Ben Ghorbel	Pilotage technique global	Architecture, intégration front/back, corrections critiques, déploiement, coordination livrables
Rindra	Support conception et implémentation	Contribution aux modules fonctionnels et validation de parcours utilisateur
Ziyad	Support backend/qualité	Aide sur stabilité, validations API et qualité de livraison
Anastasia	UX/présentation	Support structuration interface et lisibilité démo
Valerie	Documentation/soutenance	Support préparation rapport, articulation démonstration et communication

Note : les noms complets de certains membres sont à compléter dans la version finale remise.

4.2 Diagramme de Gantt (synthèse)

Lot	**S1**	**S2**	**S3**	**S4**	**S5**	**S6**
Cadrage + architecture	X	X				
Module Information		X	X			
Module Visualisation			X	X		
Module Gestion				X	X	
Module Administration					X	X
Qualité, déploiement, rapport					X	X

5 Étapes réalisées (méthodologie de projet)

5.1 Phase 1 – Planification

— Analyse du cahier des charges.
— Définition des modules et des rôles utilisateurs.
— Choix de la stack technique et de la stratégie de données.

5.2 Phase 2 – Conception

— Conception du modèle métier UML.
— Conception de l'API REST et des DTO.
— Structuration des écrans côté frontend.

5.3 Phase 3 – Développement itératif

— Implémentation incrémentale des modules Information, Visualisation, Gestion, Administration.
— Ajout de contrôles spécialisés par type d'objet (ex. machine à café, aspirateur, alarme).
— Intégration des scénarios domotiques et du suivi maintenance.

5.4 Phase 4 – Validation et stabilisation

— Vérification des builds backend et frontend.
— Correctifs UX (mise à jour immédiate des états après action).
— Construction d'une démo reproductible (seed réaliste + smoke test).

6 Couverture fonctionnelle par module

6.1 Module Information

— Consultation des objets et pièces.
— Recherche multi-critères (au moins deux filtres).
— Entrée utilisateur vers l'inscription.

6.2 Module Visualisation

— Authentification et session utilisateur.
— Consultation/édition profil.
— Consultation des objets et services avec détails enrichis.
— Mécanique de points et progression de niveau.

6.3 Module Gestion

— CRUD objets connectés.
— Contrôle d'état, configuration par pièce, paramètres spécifiques.
— Statistiques, historique des actions.
— Maintenance (détection objets à réparer, marquage réparé).
— Exports et supervision opérationnelle.

6.4 Module Administration

— Gestion des utilisateurs et des droits admin.
— Traitement des demandes de suppression.
— Protection des routes sensibles.

7 UML amélioré

7.1 Structure conceptuelle

Le modèle UML a été enrichi pour couvrir les familles d'objets connectés demandées, avec héritage et spécialisations métier.

Bloc UML	Rôle
Maison – Pièce	Une maison possède plusieurs pièces (Salon, Cuisine, Chambre, etc.)
ObjetConnecte (abstrait)	Racine commune (nom, état, connectivité, batterie, maintenance, pièce)
Branches ObjetConnecte	Ouvrant, Capteur, Appareil, BesoinAnimal
Feuilles métier	Exemples : Porte, Volet, Thermostat, Camera, DetecteurMouvement, LaveLinge, Alarme, Aspirateur, MachineCafe, Nourriture, Eau
Utilisateur (abstrait)	ParentFamille, Enfant, VoisinVisiteur + attributs de niveau/points/admin
HistoriqueAction	Trace les actions utilisateur sur les objets
Scenario + ScenarioAction	Automatisation (manuel/planifié/conditionnel)
DemandeSuppression	Workflow de suppression validé par admin

7.2 Cardinalités clés

- 1 Maison – N Pièces
- 1 Pièce – N ObjetsConnectés
- 1 Utilisateur – N HistoriqueAction
- 1 ObjetConnecte – N HistoriqueAction
- 1 Scenario – N ScenarioAction
- 1 Utilisateur – N DemandeSuppression

8 Schéma de base de données amélioré

8.1 Choix de modélisation relationnelle

- Stratégie d'héritage `SINGLE_TABLE` pour `ObjetConnecte` et `Utilisateur`.
- Clés techniques auto-générées (`id` bigint).
- Clés étrangères explicites pour garantir l'intégrité référentielle.

8.2 Tables principales

Table	Clé primaire	Contenu principal
maison	id	Nom de la maison
piece	id	Pièces + FK vers maison
utilisateur	id	Profil, points, niveau, rôle admin, type membre
objet_connecte	id	Données communes + colonnes spécialisées par type
historique_action	id	Action, timestamp, FK utilisateur, FK objet
donnee_capteur	id	Série temporelle de mesures capteurs
scenario	id	Définition scénario, type, trigger, activation
scenario_action	id	Actions élémentaires d'un scénario, FK scénario, FK objet
demande_suppression	id	Workflow de demande suppression + statut + auteur

8.3 Améliorations proposées du schéma

1. Indexer les colonnes de recherche fréquentes :

— `objet_connecte(type_objet, piece_id, etat, marque)`
— `historique_action(utilisateur_id, objet_id, timestamp)`
— `scenario(enabled, type, trigger_event)`

2. Contraintes d'unicité utiles :
   — `utilisateur.email` unique,
   — éventuellement `objet_connecte(nom, piece_id)` unique selon règle métier.
3. Journaliser les exports/administrations sensibles dans `historique_action`.

9 Tests, performance, responsive, accessibilité

9.1 Vérifications techniques

— `mvn -DskipTests compile` : succès.
— `npm.cmd run build` : succès.
— Smoke test fonctionnel : `scripts/demo-smoke.ps1`.

9.2 Responsive et UX

— Vérifications desktop/tablette/mobile.
— Interface orientée feedback (toasts, états lisibles, contrôles par type d'objet).
— Navigation modulaire claire (Information / Visualisation / Gestion / Administration).

9.3 Accessibilité (base)

— Structure sémantique globale.
— Focus visible et interactions clavier sur les actions principales.
— Contrastes et hiérarchie de lecture améliorés.

10 Déploiement et exploitation

10.1 Render

— Service API : `dev-web-ing1-api`
— Service Frontend : `dev-web-ing1-web`
— Healthcheck : `/api/health`

10.2 Important pour la soutenance

L'URL racine de l'API (`/`) retourne une 404 Whitelabel. C'est normal pour une API-only. Le point d'entrée de supervision est `/api/health`.

11 Difficultés rencontrées et solutions

1. **Évolutions du modèle et compatibilité données** : correction de problèmes de valeurs legacy, validations renforcées.
2. **Synchronisation front/back en temps réel** : mise à jour locale immédiate après `PUT` pour éviter l'impression de non-réactivité.
3. **Fiabilité démo** : seed orienté soutenance (objets à réparer garantis, scénarios disponibles, données cohérentes).

12 Conclusion et perspectives

Le projet répond au cahier des charges sur les axes techniques, fonctionnels et méthodologiques. La plateforme est utilisable en démonstration temps réel, structurée pour évoluer et prête à une version plus industrialisée.

Perspectives

- Passage complet MySQL avec migration contrôlée.
- Renforcement des tests automatiques (intégration backend et E2E frontend).
- Amélioration de l'observabilité (métriques, logs structurés, audit enrichi).
- Renforcement des contrôles de sécurité et des workflows administratifs.

Annexe A – Checklist de conformité au cahier des charges

- Rapport (<= 15 pages) : présent.
- Introduction / contexte / choix techniques : présent.
- Répartition des tâches + Gantt : présent.
- Étapes réalisées (méthodologie) : présent.
- Conclusion + perspectives : présent.
- Utilisation Git : présent.
- Frameworks : présent.
- BDD : présente.
- Responsive, accessibilité, tests : documentés.

Annexe B – Commandes utiles

```
# backend
mvn spring-boot:run

# frontend
cd frontend
npm.cmd run dev

# build
mvn -DskipTests compile
cd frontend && npm.cmd run build

# smoke demo
powershell.exe -ExecutionPolicy Bypass -File scripts\demo-smoke.ps1
```